Anagrams (recursion)

CodEx — Exercices de programmation

Two strings of the same length are anagrams if one can be written using all the characters of the other, possibly moved : `chien` and `niche` are anagrams, `louve` and `poule` are not.

We use a recursive approach : if both strings have length 1, return `True` or `False` depending on whether they are equal ; otherwise, test whether the first character of the first string also occurs in the second : if so, start again on both strings with the first occurrence of that character removed ; if not, return `False`.

Write two functions (the strings are non-empty) :

— `supprime_premier(chaine, cible)` returns a pair `(present, chaine_sans_cible)` : `present` tells whether the character `cible` is in `chaine`, and `chaine_sans_cible` is the string without the first occurrence of `cible` (or the same string if it does not occur) ;

— `anagrammes(chaine1, chaine2)` returns `True` if the two strings are anagrams, `False` otherwise.

Examples : `supprime_premier('ukulélé', 'u')` is `(True, 'kulélé')`, `supprime_premier('ukulélé', 'é')` is `(True, 'ukullé')`, `supprime_premier('ukulélé', 'a')` is `(False, 'ukulélé')` ; `anagrammes('chien', 'niche')` and `anagrammes('énergie noire', 'reine ignorée')` are `True`, `anagrammes('louve', 'poule')` is `False`.